

Uart_Esp32 - Projektdokumentation - Firmware - v0.20

Projektname: Uart_Esp32 **Dokumentenart:** Projektdokumentation **Thema:** UART Image-Transfer mit Snapshot-Dateiquellen **Erstellungsdatum:** 2026-09-21 **Autor / verantwortliche Person:** OpenAI ChatGPT, technische Bearbeitung **Version:** v0.20 **Status:** Entwurf - Host-Syntaxpruefung erfolgreich, PlatformIO- und Hardwaretest offen **Ablageort:** Uart_ESP32/docs **Referenzen:** Projektstand v0.19, BX3-UART-Konsole, Dokumentationsrichtlinie fuer PDF/DOCX und Codepakete

1. Ziel der Aenderung

In v0.19 konnte der UART Image-Transfer read-only MTD-Quellen wie `/dev/mtd7ro` uebertragen. Beim Live-Lesen einer MTD-Partition kann die Gesamt-SHA256-Pruefung jedoch fehlschlagen, wenn sich der Inhalt des laufenden Systems waehrend des Transfers veraendert.

v0.20 ergaenzt deshalb einen sicheren Snapshot-Dateiquellen-Modus. Die BX3-Konsole kann zuerst eine stabile Datei erzeugen, zum Beispiel `/tmp/mtd7.img` oder `/tmp/mtd7.img.gz`. Anschliessend uebertraegt der ESP32 nicht mehr das veraenderliche Live-Device, sondern die fertige Snapshot-Datei.

2. Umsetzung in der Firmware

Die bestehende Image-Transfer-Architektur bleibt erhalten. Es wird weiterhin blockweise mit 32 KiB gearbeitet. Pro Block liefert der BX3-Blockhelfer Groesse, POSIX-CRC32 und Base64-Daten. Der ESP32 prueft Groesse und CRC32, der Browser bestaetigt jeden Block, und bei einem Volltransfer ab Block 0 wird am Ende ein Gesamt-SHA256 verglichen.

Neu ist die Quellenvalidierung. Erlaubt sind jetzt:

Quellentyp	Beispiele	Zweck
Live-MTD	<code>/dev/mtd7ro</code>	Direkter Transfer einer read-only MTD-Quelle
Snapshot-Datei	<code>/tmp/mtd7.img</code>	Transfer einer zuvor erzeugten unkomprimierten Datei
Komprimierter Snapshot	<code>/tmp/mtd7.img.gz</code>	Transfer einer kleineren gzip-Datei
Alternative tmp-Ablage	<code>/var/tmp/mtd7.img.gz</code>	Kompatibel, wenn <code>/var/tmp</code> auf <code>/tmp</code> verweist

Die Dateiquellen sind bewusst auf `/tmp` und `/var/tmp` sowie auf die Endungen `.img` und `.img.gz` beschaenkt. Der Pfad darf nur Buchstaben, Zahlen, `/`, `_`, `-` und `.` enthalten. Dadurch wird verhindert, dass das Webinterface beliebige Shell-Pfade oder Shell-Metazeichen in den Blockhelfer einschleust.

3. Ablauf mit Snapshot-Datei

1. In der UART-Konsole wird auf dem BX3 eine Snapshot-Datei erzeugt.
2. Die Datei wird mit `ls -l` und `sha256sum` geprueft.
3. Auf der Seite UART Image-Transfer wird die Datei als Quelle eingetragen, zum Beispiel `/tmp/mtd7.img.gz`.
4. Die Firmware ermittelt die Dateigroesse mit `wc -c`.

5. Der vorhandene Blockhelfer `bx3img_block` N liest die Datei mit `dd if=<quelle> ...`
6. Der Browser sammelt die validierten Bloেকে und erzeugt den Download.
7. Der Dateiname des Downloads entspricht bei Snapshot-Quellen der Quelldatei, zum Beispiel `mtd7.img.gz`.

4. Empfohlene Konsolenbefehle

Unkomprimierter Snapshot:

```
dd if=/dev/mtd7ro of=/tmp/mtd7.img bs=32768 && sync && ls -l /tmp/mtd7.img && sha256sum /tmp/mtd7.img
```

Komprimierter Snapshot:

```
dd if=/dev/mtd7ro bs=32768 2>/dev/null | gzip -c > /tmp/mtd7.img.gz && sync && ls -l /tmp/mtd7.img.gz && sha256sum /tmp/mtd7.img.gz
```

Aufräumen nach erfolgreichem Download:

```
rm -f /tmp/mtd7.img /tmp/mtd7.img.gz /tmp/bx3blk && sync && df -h /tmp
```

5. Geänderte Dateien

Datei	Änderung
src/ConfigDefaults.h	Firmware-Defaultversion auf 0.20.0 gesetzt
src/main.cpp	Build-Anzeige auf v0.20, Image-Transfer-Seite fuer Snapshot-Dateiquellen erweitert, Downloadname angepasst
src/UartManager.h	Image-Transfer-Kommentar und interner Dateiquellenstatus ergaenzt
src/UartManager.cpp	Quellenvvalidierung, Dateigroessenermittlung per <code>wc -c</code> , Datei-/MTD-Pfadlogik und Statusmeldungen ergaenzt
README.md	Stand v0.20 und Snapshot-Dateiquellen dokumentiert
VALIDATION_v0.20.txt	Validierungsprotokoll ergaenzt

6. Sicherheits- und Bedienlogik

Die bestehende Sicherheitslogik bleibt erhalten. Der Image-Transfer reserviert UART TX/RX, waehrend er aktiv ist. Nach einem Fehler wird die UART-Konsole wieder freigegeben. Ein Neustart bleibt gesperrt, bis der Benutzer den Shell-Zugriff in der Konsole wiederhergestellt und dort die Freigabe bestaetigt hat.

Der neue Snapshot-Dateimodus erweitert nur die erlaubten Quellen. Er hebt die TX-Sperren, den Probe-Runner-Ausschluss und die Neustartlogik nicht auf.

7. Validierung

Durchgefuehrt wurden:

- JavaScript-Syntaxpruefung aller eingebetteten Webskripte mit `node --check`.
- C++17-Syntaxpruefung von `UartManager.cpp` und `TestCandidateCatalog.cpp` mit lokalen ESP32/Arduino-Host-Stubs.

- Prüfung der ZIP-Struktur und der geänderten Dateien.
- DOCX- und PDF-Erzeugung mit visueller Kontrolle der geänderten Seiten.

Nicht durchgeführt wurden:

- echter PlatformIO-Build,
- Flashen auf einen ESP32,
- realer BX3-End-to-End-Transfer von /tmp/mtd7.img.gz.

8. Offene Punkte

Vor dem produktiven Einsatz sollte `pio run -e uart_esp32_usb` erfolgreich durchlaufen. Danach sollte ein realer Test mit einer komprimierten Snapshot-Datei erfolgen. Auf dem PC muss der SHA256 der heruntergeladenen Datei mit dem in der BX3-Konsole ausgegebenen SHA256 identisch sein.

9. Änderungshistorie

Version	Datum	Status	Änderung
v0.17	2026-09-21	Entwurf	UART Image-Transfer und kopierbare Konsole eingefuehrt
v0.18	2026-09-21	Entwurf	Browser-/ACK-Synchronisierung und Neustartlogik nach Unterbrechung korrigiert
v0.19	2026-09-21	Entwurf	CRC32 pro Block, groesserer RX-Puffer, schnellerer RX-Pfad und Blockhelfer eingefuehrt
v0.20	2026-09-21	Entwurf	Snapshot-Dateien aus /tmp und /var/tmp als Image-Transfer-Quelle erlaubt